

**МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ**

КУРСОВАЯ РАБОТА

по дисциплине «Введение в нереляционные базы данных»

**Тема: Биометрическая система контроля и управления доступом
(СКУД) для университета**

Студент гр. 3388	_____	Березовский М.А.
Студент гр. 3388	_____	Потапов Р.Ю.
Студент гр. 3388	_____	Шубин П.А.
Преподаватель	_____	Заславский М.М.

Санкт-Петербург
2026

ЗАДАНИЕ

Студенты гр. 3388

Березовский М.А.

Потапов Р.Ю.

Шубин П.А.

Тема проекта:

Разработка прототипа биометрической системы контроля и управления доступом (СКУД) для университета.

Исходные данные:

Необходимо реализовать веб-приложение для хранения биометрических данных (лицо, голос), управления доступом пользователей к зонам университета и журналирования всех связанных событий. СУБД: MongoDB.

Содержание пояснительной записки:

- «Содержание»
- «Введение»
- «Качественные требования к решению»
- «Сценарии использования»
- «Модель данных»
- «Разработанное приложение»
- «Вывод»
- «Приложения»

Предполагаемый объём пояснительной записки:

Не менее 15 страниц.

Дата выдачи задания: _____

Дата сдачи: _____

Студенты

Березовский М.А.

Потапов Р.Ю.

Шубин П.А.

Преподаватель

Заславский М.М.

АННОТАЦИЯ

В рамках данного курса разрабатывался прототип биометрической системы контроля и управления доступом (СКУД) для университета. Система реализует хранение профилей пользователей (студентов, сотрудников, гостей) и их биометрических шаблонов (лицо, голос), управление иерархией групп и правами доступа к зонам университета, регистрацию всех событий прохода, а также инструменты аналитики и массового импорта/экспорта данных.

В качестве СУБД использована документно-ориентированная система MongoDB, что обусловлено необходимостью хранить сложные вложенные структуры (биометрические шаблоны внутри профиля пользователя) и обрабатывать быстрорастущий поток событий прохода. Backend реализован на Python с использованием фреймворка FastAPI. Приложение развёртывается через Docker Compose.

Исходный код и документация доступны по адресу:
<https://github.com/moevm/nsql1h26-bio>

ANNOTATION

In the course of this project, a prototype biometric access control and management system (ACS) for a university was developed. The system stores user profiles (students, staff, and guests) along with their biometric templates (face and voice), manages hierarchical user groups and zone access policies, logs all entry events, and provides analytics tools with bulk import/export functionality.

MongoDB was chosen as the database system, which is well-suited for storing complex nested structures (biometric templates embedded inside user profiles) and handling a rapidly growing stream of access events. The backend is

implemented in Python using the FastAPI framework. The application is deployed via Docker Compose.

Source code and documentation are available at:
<https://github.com/moevm/nsql1h26-bio>

СОДЕРЖАНИЕ

1. Введение
 - 1.1 Актуальность
 - 1.2 Постановка задачи
 - 1.3 Предлагаемое решение
2. Качественные требования к решению
3. Сценарии использования
 - 3.1 Макет пользовательского интерфейса
 - 3.2 Сценарий 1: Управление карточкой пользователя
 - 3.3 Сценарий 2: Настройка уровней доступа
 - 3.4 Сценарий 3: Поиск и аудит событий
 - 3.5 Сценарий 4: Массовый импорт и экспорт данных
 - 3.6 Сценарий 5: Кастомизируемая аналитика
4. Модель данных
 - 4.1 Нереляционная модель данных (MongoDB)
 - 4.2 Реляционная модель данных (SQL)
 - 4.3 Сравнение моделей
 - 4.4 Вывод по модели данных
5. Разработанное приложение
 - 5.1 Архитектура
 - 5.2 Используемые технологии
 - 5.3 Маршруты API
 - 5.4 Снимки экрана приложения
6. Вывод
- Литература
- Приложения

1. ВВЕДЕНИЕ

1.1 Актуальность

Системы контроля и управления доступом (СКУД) являются неотъемлемой частью инфраструктуры безопасности современных учреждений. Традиционные решения на основе карт-пропусков имеют существенные недостатки: карту можно потерять, передать другому лицу или подделать. Биометрические системы лишены этих недостатков — они идентифицируют человека по уникальным физиологическим признакам, которые нельзя передать или забыть.

Для университета, где ежедневно перемещаются тысячи студентов и сотрудников, актуальна задача автоматизированного, точного и масштабируемого контроля доступа. При этом система должна хранить большие объёмы биометрических данных (векторные эмбединги лиц и голосов), обрабатывать непрерывный поток событий прохода и предоставлять удобные инструменты аудита и аналитики.

1.2 Постановка задачи

Требуется разработать прототип веб-приложения — биометрической СКУД для университета — со следующими основными функциями:

- хранение профилей пользователей (студенты, сотрудники, гости);
- хранение биометрических шаблонов (лицо, голос) в виде векторных эмбедингов;
- управление группами пользователей и иерархией доступа;
- управление зонами университета и устройствами контроля доступа;
- настройка политик доступа (кто, куда, когда);
- регистрация и журналирование всех событий прохода;
- аналитика: статистика проходов, топ зон по отказам, распределение по времени;
- массовый импорт и экспорт всех данных системы в формате JSON.

1.3 Предлагаемое решение

Разрабатывается веб-приложение на базе Python (FastAPI) с нереляционной СУБД MongoDB. Приложение развёртывается через Docker Compose. Веб-интерфейс предоставляет единую точку входа для всех ролей пользователей. Аутентификация реализована через JWT-токены (логин и пароль).

Выбор MongoDB обусловлен предметной областью: данные системы естественно представляются в виде документов с вложенными структурами — профиль пользователя содержит массив биометрических шаблонов, политика доступа содержит список зон. Кроме того, MongoDB хорошо масштабируется горизонтально, что критично для быстрорастущей коллекции событий прохода.

2. КАЧЕСТВЕННЫЕ ТРЕБОВАНИЯ К РЕШЕНИЮ

Система должна принимать решение о допуске на основе актуальных политик, расписания и статуса пользователя. Количество обращений к базе данных при проверке доступа должно быть минимальным. Журнал событий должен сохранять данные о пользователе, зоне и устройстве на момент прохода - вне зависимости от последующих изменений в системе. Поиск и фильтрация по журналу должны быть доступны через веб-интерфейс. Приложение должно запускаться командой `docker compose up` без дополнительной настройки. Тестовые данные должны быть синтетическими и не содержать персональной информации реальных людей.

Поскольку каждый проход порождает одну запись, но требует нескольких операций чтения (профиль пользователя, устройство, политики), а аналитика и аудит целиком основаны на чтении, в системе преобладают операции чтения над операциями записи.

3. СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ

3.1 Макет пользовательского интерфейса

Приложение построено как единое веб-приложение со следующими основными экранами, навигация между которыми выполняется через боковое меню. Все поисковые фильтры кодируются в URL страницы (query-параметры), что позволяет сохранять и передавать ссылки на результаты поиска.

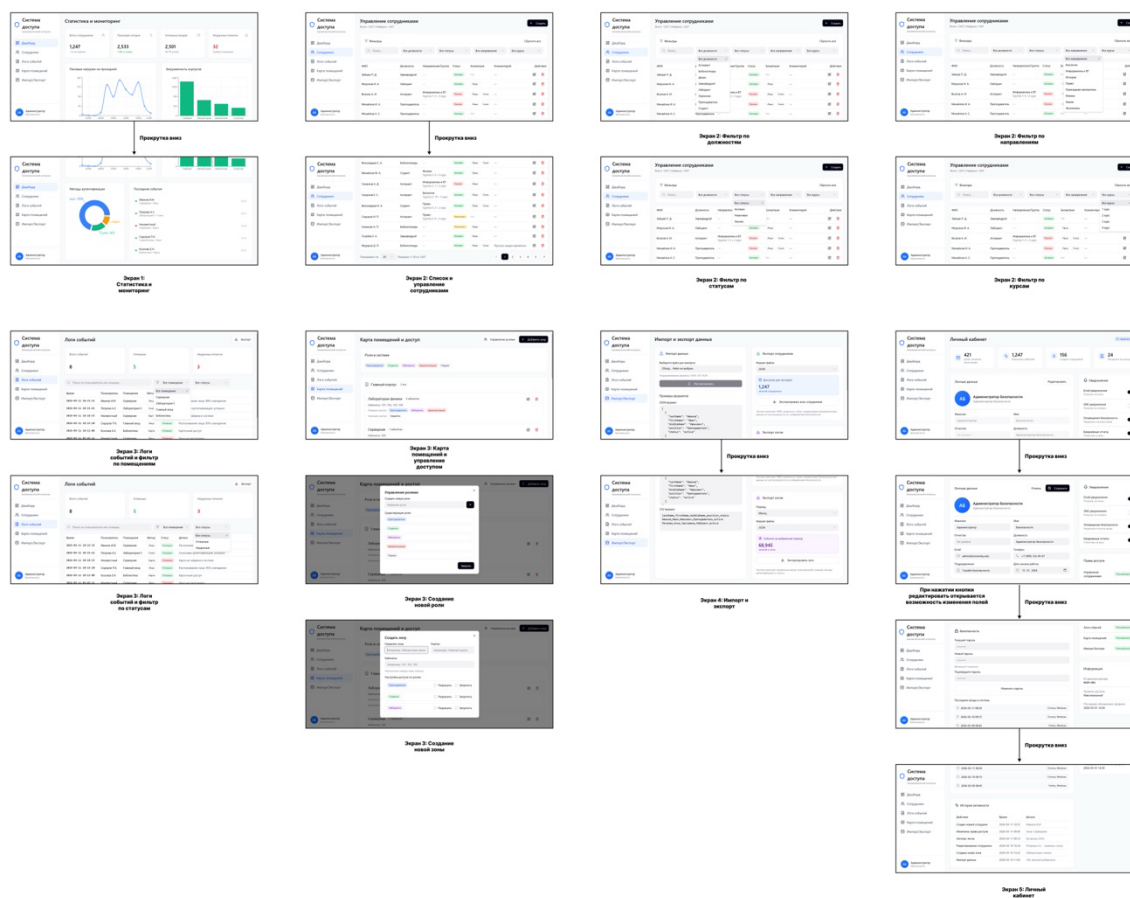


Рисунок 1 — Макет пользовательского интерфейса — схема экранов приложения

Основные страницы приложения и их адреса:

- Страница входа (/login) — форма авторизации по логину и паролю.
- Дашборд (/) — сводная статистика: проходы за сутки, топ зон, процент отказов.

- Пользователи (/people) — таблица с фильтрацией по ФИО, роли, отделу и статусу.
- Карточка пользователя (/people/{id}) — просмотр и редактирование профиля, биометрических шаблонов, истории проходов.
- Группы (/groups) — иерархический список групп с возможностью создания и редактирования.
- Зоны (/zones) — список контролируемых зон университета.
- Устройства (/devices) — список терминалов и турникетов с привязкой к зонам.
- Политики доступа (/policies) — список правил с фильтрацией по субъекту и зоне.
- Журнал событий (/events) — таблица с многокритериальным поиском и серверной пагинацией.
- Аналитика (/analytics) — интерактивные графики с кастомизируемыми фильтрами.
- Импорт / Экспорт (/maintenance) — кнопки массового импорта и экспорта всей базы данных.

3.2 Сценарий 1: Управление карточкой пользователя (MVP)

Актёр: Администратор безопасности. Цель: добавить нового человека в систему, загрузить биометрию или изменить данные. Предусловие: администратор авторизован в системе.

Основной поток:

1. Администратор открывает страницу /people.
2. Нажимает кнопку «Создать».
3. Заполняет поля: ФИО, роль (студент / сотрудник / гость), факультет / отдел, статус.
4. Загружает файл с биометрическим шаблоном (face или voice embedding).
5. Сохраняет карточку — система подтверждает создание.

Альтернативный поток: редактирование существующего пользователя — смена статуса на «Заблокирован», добавление нового биометрического шаблона или удаление устаревшего.

Данный сценарий реализован в прототипе в полном объёме.

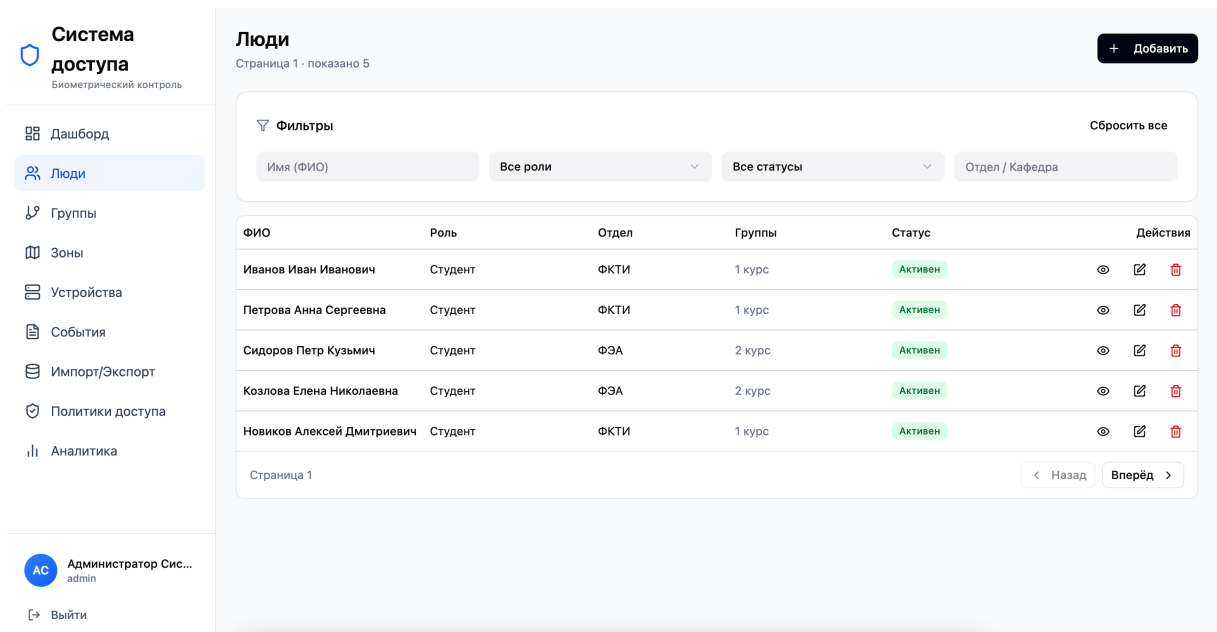


Рисунок 2 — Экран списка пользователей с многокритериальным поиском

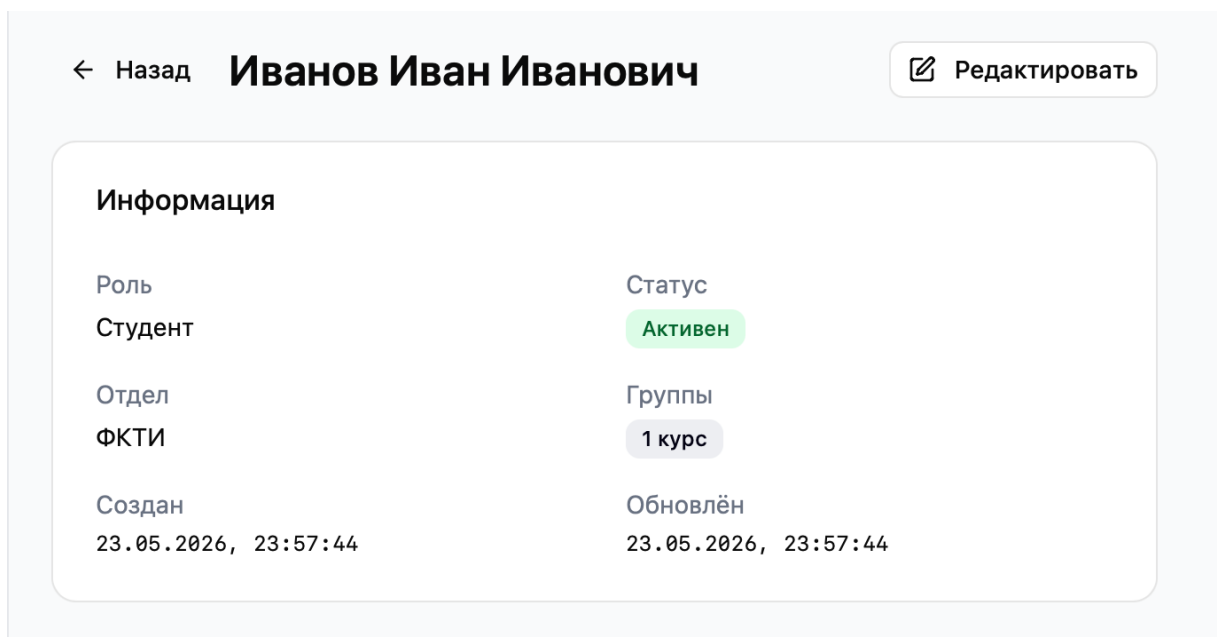


Рисунок 3 — Карточка пользователя с биометрическими шаблонами

3.3 Сценарий 2: Настройка уровней доступа

Актёр: Администратор безопасности. Цель: ограничить доступ группы пользователей к определённым зонам.

Основной поток:

1. Администратор открывает страницу /policies.
2. Нажимает «Создать политику».
3. Выбирает субъект политики: группа или конкретный пользователь.

4. Выбирает одну или несколько зон, к которым применяется политика.
5. Устанавливает расписание (дни недели, временной интервал) и срок действия.
6. Сохраняет политику. Система немедленно применяет её при следующей попытке прохода.

Данный сценарий реализован в прототипе в полном объёме.

3.4 Сценарий 3: Поиск и аудит событий (MVP)

Актёр: Охранник / Аудитор. Цель: узнать, кто входил в серверную в ночное время.

Основной поток:

1. Пользователь открывает страницу /events.
2. Устанавливает фильтры: зона («Серверная»), временной диапазон, результат («DENY» или «ALLOW»).
3. Система отображает список событий с серверной пагинацией.
4. Пользователь нажимает на строку события — открывается детальная информация: метод биометрии, скор распознавания, имя пользователя и устройство.

Данный сценарий реализован в прототипе в полном объёме.

Система доступа
Биометрический контроль

Логи событий
Страница 1 · показано 10

Фильтры

Зона (ID) 24.05.2026, 12:30 24.05.2026, 12:30 Все

Время	Человек	Зона	Метод	Решение	Score
23.05.2026, 23:52:44	Иванов Иван Иванович	Главный вход	face	ALLOW	0.97
23.05.2026, 23:45:44	Петрова Анна Сергеевна	Главный вход	face	ALLOW	0.93
23.05.2026, 23:37:44	Сидоров Петр Кузьмич	Лаборатория 3	face	DENY	0.41
23.05.2026, 23:22:44	Козлова Елена Николаевна	Библиотека	voice	ALLOW	0.88
23.05.2026, 23:07:44	Новиков Алексей Дмитриевич	Главный вход	face	ALLOW	0.95
23.05.2026, 23:02:44	Морозов Дмитрий Александрович	Лаборатория 3	face	ALLOW	0.99
23.05.2026, 22:37:44	Волкова Мария Павловна	Лаборатория 5	voice	ALLOW	0.91
23.05.2026, 22:22:44	Зайцев Андрей Викторович	Серверная	face	ALLOW	0.96
23.05.2026, 22:07:44	Лебедева Ольга Игоревна	Главный вход	card	DENY	0.3
23.05.2026, 21:52:44	Иванов Иван Иванович	Лаборатория 3	face	DENY	0.38

Страница 1

< Назад Вперёд >

Рисунок 4 — Журнал событий прохода с многокритериальным поиском

3.5 Сценарий 4: Массовый импорт и экспорт данных (MVP)

Актёр: Администратор БД. Цель: загрузить список пользователей или выгрузить все данные системы для резервного копирования.

Основной поток (экспорт):

1. Администратор открывает страницу /maintenance.
2. Нажимает кнопку «Экспорт».
3. Браузер скачивает единый JSON-файл со всеми коллекциями базы данных.

Основной поток (импорт):

4. Администратор выбирает ранее экспортированный JSON-файл.
5. Нажимает кнопку «Импорт».
6. Система очищает базу данных и восстанавливает её из файла.
7. Отображается сообщение о количестве восстановленных записей по каждой коллекции.

Данный сценарий реализован в прототипе в полном объёме.

3.6 Сценарий 5: Кастомизируемая аналитика

Актёр: Администратор / Аудитор. Цель: оценить загруженность зон, выявить проблемные точки доступа.

Основной поток:

1. Пользователь открывает страницу /analytics.
8. Устанавливает многокритериальный фильтр: временной диапазон, зона, результат прохода, метод биометрии.
9. Выбирает параметры для осей графика (например: ось X — время суток, ось Y — количество проходов).
10. Система строит диаграмму на основе агрегации данных из коллекции access_events.

Реализованная аналитика включает: топ зон по отказам, распределение отказов по причинам, пиковые нагрузки по часам, статистику по устройствам.

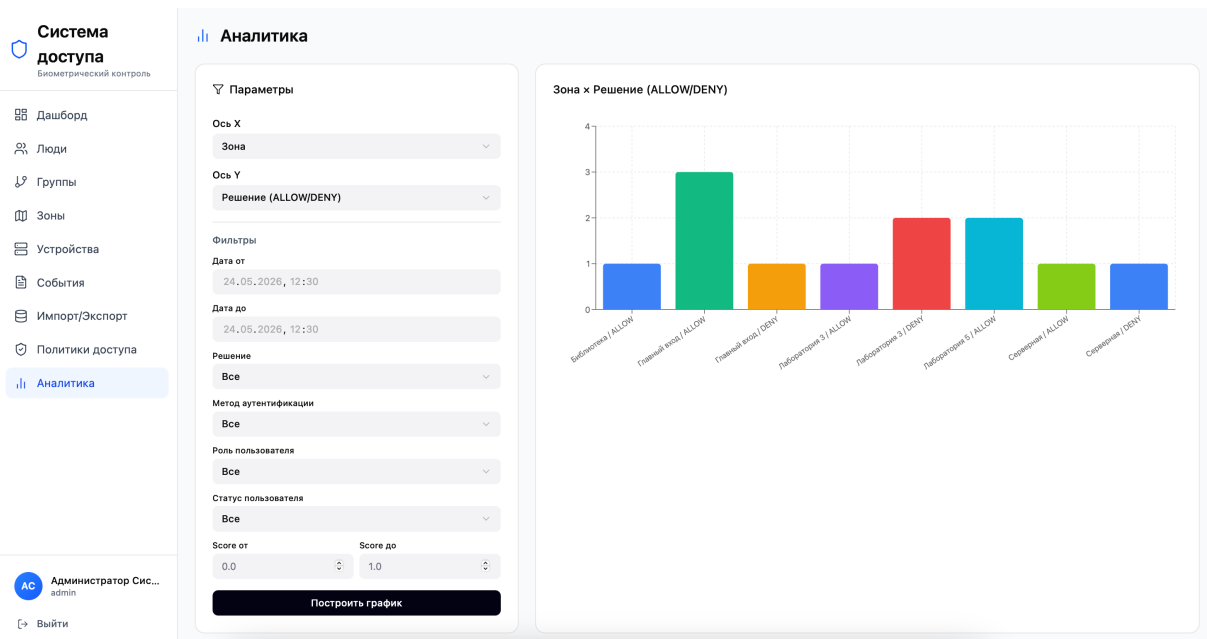


Рисунок 5 — Страница аналитики с кастомизируемыми графиками

4. МОДЕЛЬ ДАННЫХ

4.1 Нереляционная модель данных (MongoDB)

4.1.1 Выбор типа нереляционной модели

Для системы выбрана документно-ориентированная модель данных на базе MongoDB. Такой выбор обусловлен предметной областью: данные естественно представляются в виде документов с вложенными структурами — пользователь содержит биометрические шаблоны, группа имеет иерархию через ссылку на родителя, событие прохода содержит контекст попытки доступа.

Основные принципы модели: биометрические шаблоны вложены в документ пользователя (читаются одним запросом); связи между сущностями хранятся через ObjectId-ссылки; журнал событий денормализован — содержит снимки имён пользователя, зоны и устройства для исторической корректности и быстрой аналитики.

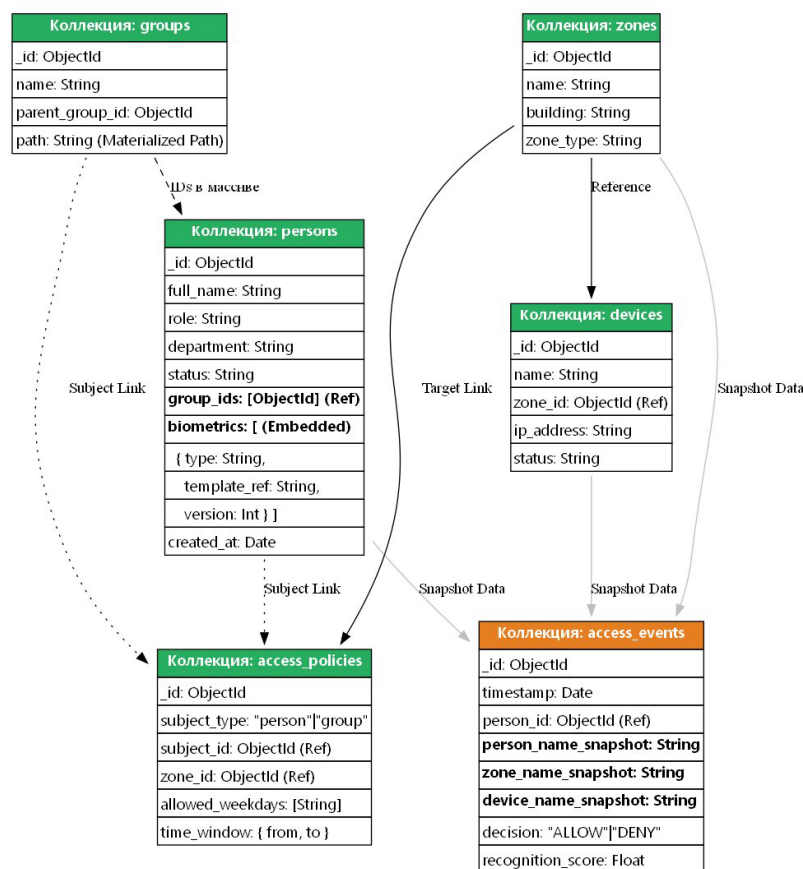


Рисунок 6 — ER-диаграмма нереляционной модели данных

4.1.2 Состав и назначение коллекций

В модели используется шесть коллекций. Ниже приведено описание назначения каждой и оценка размера одного документа в байтах.

Таблица 1 — Состав коллекций нереляционной модели

Коллекция	Назначение	Размер документа (байт)
persons	Профили пользователей с вложенными биометрическими шаблонами	$100 + 12 \cdot k_g + 2160 \cdot k_b$
groups	Иерархия групп университета (связь через parent_group_id)	136
zones	Описание контролируемых зон	77
devices	Устройства контроля доступа с привязкой к зонам	88
access_policies	Правила доступа: субъект, зоны, расписание, срок	$58 + 12 \cdot k_z + 3 \cdot k_w$
access_events	Журнал всех попыток прохода (денормализован)	205

Где: k_g — число групп у пользователя, k_b — число биометрических шаблонов, k_z — число зон в политике, k_w — число разрешённых дней недели. Один биометрический шаблон занимает ~2160 байт, из которых ~2048 байт — векторный эмбединг (512×4 байта float32).

4.1.3 Оценка объёма информации

Пусть P — число пользователей, G — групп, Z — зон, D — устройств, A — политик, E — событий прохода. При типичных параметрах: $k_g_{avg} = 2$, $k_b_{avg} = 2$, $k_z_{avg} = 3$, $k_w_{avg} = 5$ формула логического объёма принимает вид:

$$V_{logic} = 4444 \cdot P + 136 \cdot G + 77 \cdot Z + 88 \cdot D + 109 \cdot A + 205 \cdot E$$

Числовой пример ($P=1000$, $G=50$, $Z=30$, $D=60$, $A=100$, $E=50000$):

$$\begin{aligned} V &= 4444 \cdot 1000 + 136 \cdot 50 + 77 \cdot 30 + 88 \cdot 60 + 109 \cdot 100 + 205 \cdot 50000 \\ &= 4\,444\,000 + 6\,800 + 2\,310 + 5\,280 + 10\,900 + 10\,250\,000 \\ &\approx 14\,719\,290 \text{ байт} \approx 14 \text{ МБ} \end{aligned}$$

Это логический объём полезных данных. Реальный размер MongoDB-базы на диске больше из-за хранения имён полей в каждом BSON-документе, индексов, служебных структур движка WiredTiger и его сжатия.

Поэтому данная оценка используется для анализа структуры модели и характера роста данных, а не для точного прогноза объёма.

4.1.4 Избыточность данных (денормализация)

Коллекция `access_events` содержит три `snapshot`-поля, намеренно дублирующие данные из справочных коллекций: `person_name_snapshot`, `zone_name_snapshot`, `device_name_snapshot`. Это осознанная денормализация, обеспечивающая быстрое чтение журнала без JOIN-аналогов и историческую корректность данных.

Размер события без `snapshot`-полей: $S_{\text{clean}} = 109$ байт. Фактический размер: $S_{\text{fact}} = 205$ байт. Коэффициент денормализации: $R = 205 / 109 \approx 1.88$.

4.1.5 Направление роста модели

Справочные коллекции (`persons`, `groups`, `zones`, `devices`, `access_policies`) растут медленно и пропорционально числу сотрудников и инфраструктуры университета. Коллекция `access_events` растёт постоянно: каждый проход порождает новую запись. При 1000 пользователях и 5 проходах в день — около 5000 записей в день, что составляет ~1,8 млн записей в год. Именно эта коллекция является доминирующим фактором роста и требует индексации, архивирования и серверной пагинации.

4.1.6 Примеры данных

Пример документа пользователя в коллекции `persons`:

```
{
  "_id": { "$oid": "67c7f10000000000000000000001" },
  "full_name": "Иванов Иван Иванович",
  "role": "student",
  "department": "ФКТИ",
  "status": "active",
  "group_ids": [{ "$oid": "67c7f1000000000000000000000101" }],
  "biometrics": [{
    "type": "face",
    "embedding": { "$binary": { ... } },
    "embedding_dimension": 512,
    "model_version": "FaceNet-v1",
    "quality_score": 0.97,
    "created_at": { "$date": "2026-03-20T10:00:00Z" }
  ]},
  "created_at": { "$date": "2026-03-20T09:55:00Z" }
```

}

Пример документа события прохода в коллекции access_events:

```
{
  "_id": { "$oid": "67c7f1000000000000000001001" },
  "timestamp": { "$date": "2026-03-21T08:14:27Z" },
  "person_id": { "$oid": "67c7f100000000000000000001" },
  "auth_method": "face",
  "decision": "ALLOW",
  "reason": "Policy matched",
  "recognition_score": 0.93,
  "person_name_snapshot": "Иванов Иван Иванович",
  "zone_name_snapshot": "Лаборатория ИИ",
  "device_name_snapshot": "Turnstile-3"
}
```

4.1.7 Примеры запросов к нереляционной модели

Таблица 2 — Запросы к нереляционной модели по основным сценариям использования

Юзкейс	MongoDB-запрос	Запросов	Коллекций
Регистрация пользователя	insertOne → persons	1	1
Добавление биометрии	updateOne (\$push) → persons	1	1
Получение политик доступа	find (\$or) → access_policies	1	1
Проверка попытки прохода	findOne + findOne + find + insertOne	4	4
История проходов	find → access_events	1	1
Аналитика отказов по зонам	aggregate (match+group+sort)	1	1

4.2 Реляционная модель данных (SQL)

4.2.1 Состав таблиц

Реляционный аналог системы включает девять таблиц, связанных через внешние ключи (FK). Таблицы `person_groups` и `policy_zones` являются связующими и реализуют отношения «многие ко многим» (M:N).

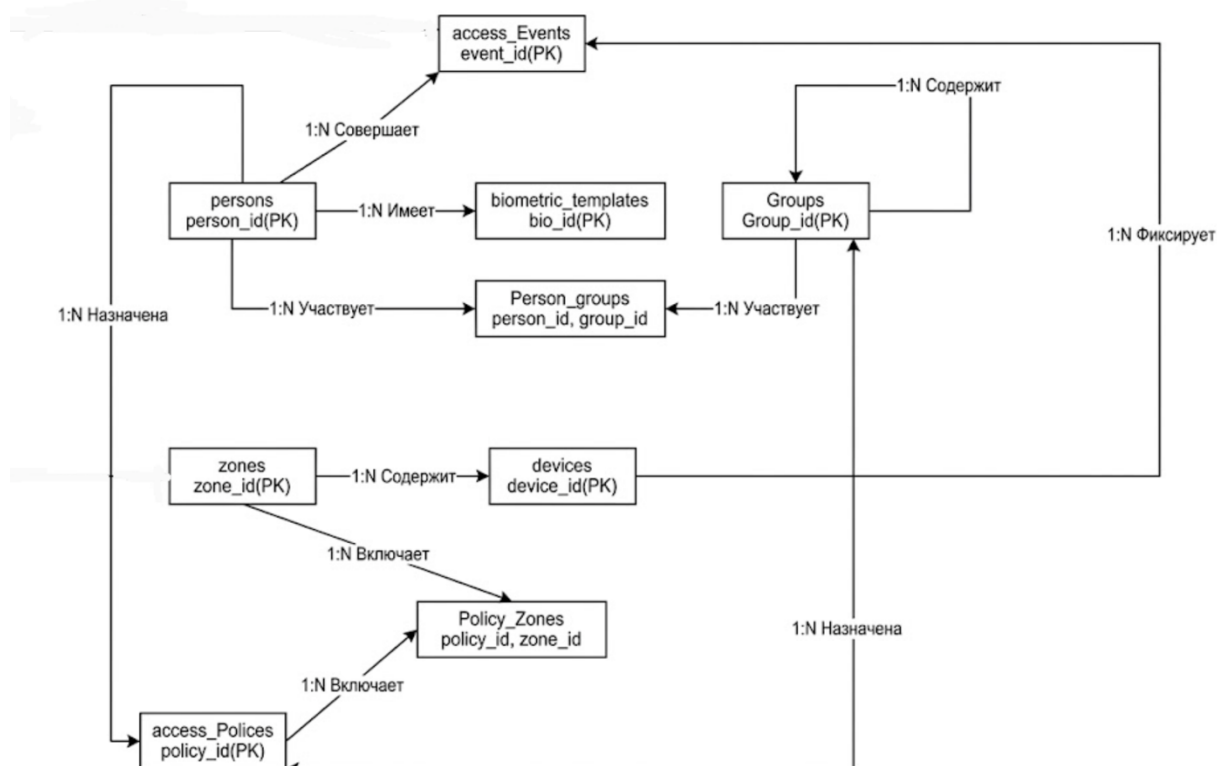


Рисунок 7 — ER-диаграмма реляционной модели данных

Таблица 3 — Состав таблиц реляционной модели

Таблица	Назначение	Размер строки (байт)
persons	Профили пользователей	222
groups	Иерархия групп	363
person_groups	Связь M:N пользователь–группа	8
biometric_templates	Биометрические шаблоны	2378
zones	Зоны университета	254
devices	Устройства контроля доступа	78
access_policies	Политики доступа	94
policy_zones	Связь M:N политика–зона	8
access_events	Журнал событий прохода	126

4.2.2 Оценка объёма

Объём данных реляционной модели при тех же параметрах (P=1000, G=50 и т.д.):

$$\begin{aligned}
 V_{\text{data}} &= 222\,000 + 18\,150 + 12\,000 + 4\,756\,000 + 7\,620 + 4\,680 \\
 &\quad + 9\,400 + 2\,400 + 6\,300\,000 = 11\,332\,250 \text{ байт} \\
 V_{\text{index}} &= 12 \times 55\,040 = 660\,480 \text{ байт} \quad (\text{B-tree, } \sim 12 \text{ байт/запись}) \\
 V_{\text{total}} &\approx 11\,992\,730 \text{ байт} \approx 11.4 \text{ МБ}
 \end{aligned}$$

4.2.3 Примеры запросов к реляционной модели

Таблица 4 — Запросы к реляционной модели по основным сценариям использования

Юзкейс	SQL-операция	Запросов	JOIN-ов
Регистрация пользователя	INSERT INTO persons	1	0
Добавление биометрии	INSERT INTO biometric_templates	1	0
Получение политик доступа	SELECT + LEFT JOIN person_groups	1	1
Проверка попытки прохода	SELECT + SELECT + SELECT + INSERT	4	1
История проходов	SELECT FROM access_events	1	0
Аналитика отказов по зонам	SELECT + JOIN devices + JOIN zones	1	2

4.3 Сравнение моделей

Ниже приведено сравнение нереляционной и реляционной моделей по ключевым критериям.

Таблица 5 — Сравнение нереляционной и реляционной моделей

Критерий	MongoDB (NoSQL)	PostgreSQL (SQL)
Логический объём данных	~14 МБ	~11.4 МБ
Причина разницы	Денормализация access_events, хранение имён полей в BSON	Нормализация, фиксированная схема
Проверка доступа	2 запроса, 0 JOIN	4 запроса, 1 JOIN
Аналитика отказов	1 агрегация, 1 коллекция	1 запрос, 2 JOIN
Гибкость схемы	Высокая: новые поля без миграции	Низкая: требует ALTER TABLE
Целостность данных	Нет FK: проверяется на уровне приложения	FK-ограничения гарантируют целостность
Масштабирование	Горизонтальное (шардирование)	Вертикальное (сложнее шардировать)

4.4 Вывод по модели данных

Для биометрической системы контроля доступа нереляционная документно-ориентированная модель (MongoDB) является предпочтительной по следующим причинам.

Производительность на чтение. Хранение «пользователь + биометрия» как единого документа исключает JOIN-операции при каждом проходе, что критично для системы реального времени.

Эффективность аналитики. Денормализованный журнал access_events позволяет строить отчёты без нагрузки на справочные коллекции — агрегации выполняются в рамках одной коллекции.

Гибкость схемы. При добавлении нового типа биометрии (например, радужка глаза) достаточно добавить новый объект в массив biometrics — не требуется миграция схемы и простой базы данных.

Горизонтальное масштабирование. MongoDB поддерживает шардирование «из коробки», что критично для быстрорастущей коллекции событий прохода.

SQL-модель предпочтительна при жёстких требованиях к целостности данных (FK-ограничения) и при небольших объёмах событий. Для данной задачи преимущества NoSQL-подхода перевешивают.

5. РАЗРАБОТАННОЕ ПРИЛОЖЕНИЕ

5.1 Архитектура

Приложение построено по трёхуровневой архитектуре с разделением ответственности между слоями.

API Layer (FastAPI) отвечает за обработку HTTP-запросов, валидацию входных данных через Pydantic-схемы, формирование ответов и аутентификацию через JWT-токены.

Service Layer реализует бизнес-логику: проверку доступа, работу с иерархией групп, объединение политик, аналитические вычисления и принятие решений ALLOW/DENY.

Repository Layer изолирует работу с MongoDB: CRUD-операции, агрегации через Aggregation Pipeline и изоляция слоя хранения данных.

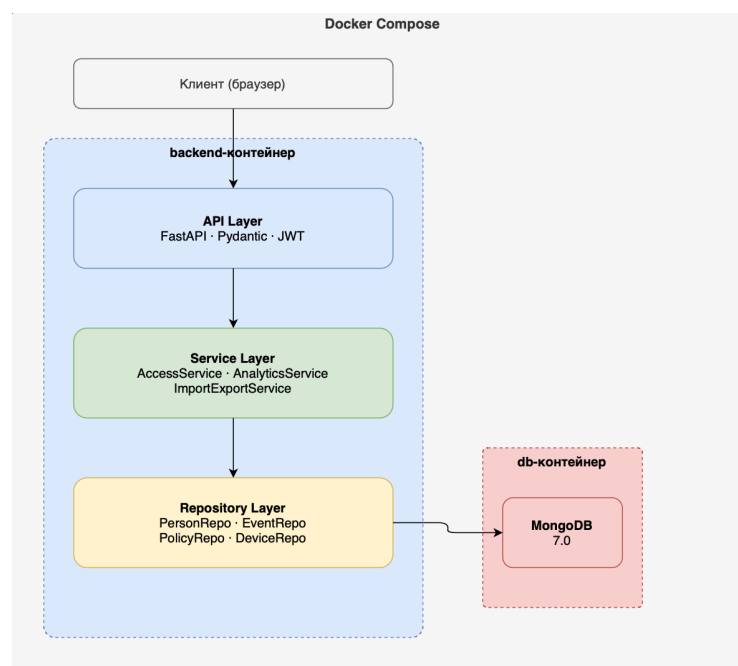


Рисунок 8 — Трёхуровневая архитектура приложения

Приложение развёртывается в Docker Compose и состоит из двух контейнеров: backend (FastAPI-приложение, порт 127.0.0.1:8080:8080) и db (MongoDB, без проброса портов наружу). При первом запуске backend автоматически проверяет состояние базы данных и, если она пуста, загружает тестовые данные из встроенного JSON-дампа.

5.2 Используемые технологии

Таблица 6 — Используемые технологии

Компонент	Технология / инструмент
СУБД	MongoDB (document-oriented, WiredTiger engine)
Backend	Python 3.11, FastAPI, Pydantic v2, Motor (async PyMongo)
Аутентификация	JWT-токены (python-jose), аутентификация по логину и паролю
Контейнеризация	Docker, Docker Compose v2
Формат импорта/экспорта	JSON (единый файл для всех коллекций)
Язык запросов	MongoDB Query Language (MQL) + Aggregation Pipeline

5.3 Основные маршруты API

Таблица 7 — Основные маршруты REST API

Метод	Путь	Описание
POST	/auth/login	Аутентификация, возврат JWT-токена
GET	/people	Список пользователей с фильтрацией и пагинацией
POST	/people	Создание профиля пользователя
GET	/people/{id}	Карточка пользователя
PUT	/people/{id}	Редактирование профиля
POST	/people/{id}/biometrics	Добавление биометрического шаблона
GET	/events	Журнал событий с фильтрацией
POST	/device/access-attempt	Попытка прохода (основной endpoint СКУД)
GET	/analytics/custom	Кастомизируемая статистика
GET	/export	Экспорт всей базы в JSON
POST	/import	Импорт базы из JSON-файла

5.4 Снимки экрана приложения

Ниже представлены основные экраны разработанного приложения.

Вход

Логин

Пароль

Войти

Рисунок 9 — Страница авторизации

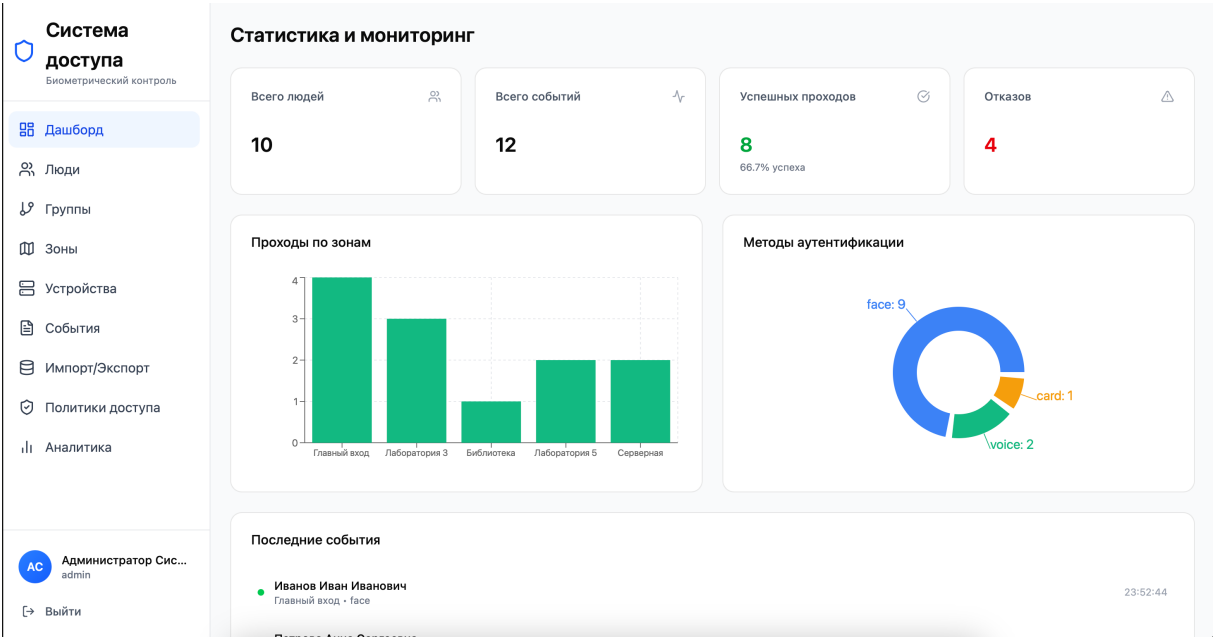


Рисунок 10 — Главная страница — сводный дашборд

Логи событий

Страница 1 · показано 4

Фильтры

Зона (ID)

24.05.2026, 12:30

24.05.2026, 12:30

DENY

Время	Человек	Зона	Метод	Решение	Score
23.05.2026, 23:37:44	Сидоров Петр Кузьмич	Лаборатория 3	face	DENY	0.41
23.05.2026, 22:07:44	Лебедева Ольга Игоревна	Главный вход	card	DENY	0.3
23.05.2026, 21:52:44	Иванов Иван Иванович	Лаборатория 3	face	DENY	0.38
23.05.2026, 21:37:44	Морозов Дмитрий Александрович	Серверная	face	DENY	0.55

Страница 1

Назад

Вперед

Рисунок 11 — Журнал событий — пример применения фильтров

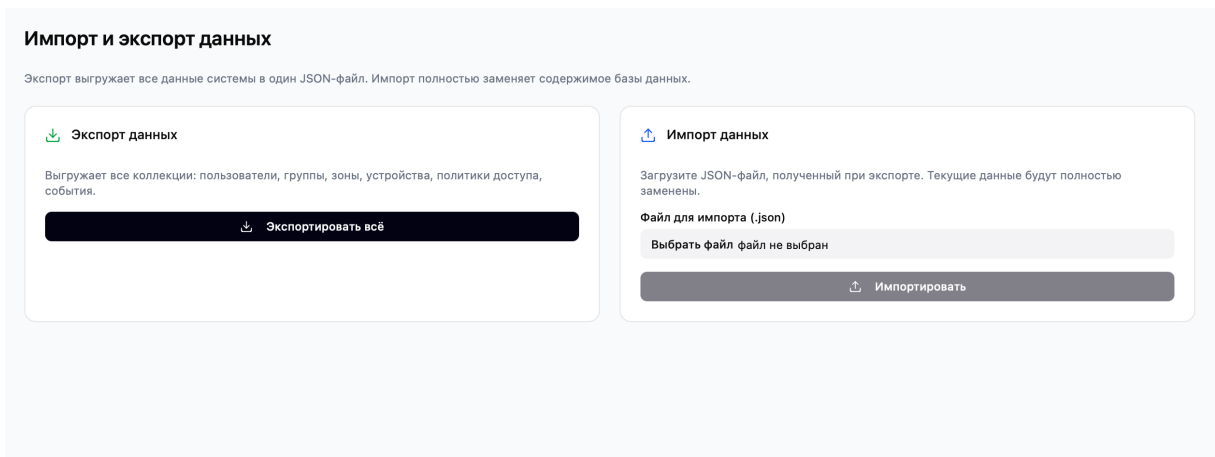


Рисунок 12 — Страница обслуживания — импорт и экспорт данных

6. ВЫВОД

6.1 Достигнутые результаты

В ходе работы разработан прототип биометрической СКУД для университета, реализующий следующие возможности:

- управление профилями пользователей (студенты, сотрудники, гости) с биометрическими шаблонами (лицо, голос);
- иерархическая модель групп с наследованием прав доступа;
- гибкие политики доступа с расписанием, сроком действия и привязкой к нескольким зонам;
- журналирование всех событий прохода с денормализованными снимками для исторической корректности;
- аналитика на базе MongoDB Aggregation Pipeline, включая кастомизируемую статистику;
- массовый импорт и экспорт всей базы данных одним действием в формате JSON;
- развёртывание через Docker Compose с автоматической загрузкой тестовых данных.

Проведено сравнение нереляционной и реляционной моделей данных.

Показано, что для данной предметной области MongoDB предпочтительна по производительности операций чтения, гибкости схемы и возможности горизонтального масштабирования.

6.2 Будущее развитие

Интеграция с реальными алгоритмами биометрического распознавания (FaceNet, ECAPA-TDNN). Добавление WebSocket-канала для оповещения охраны о событиях в режиме реального времени. Версионирование биометрических шаблонов. Поддержка дополнительных типов биометрии (радужка глаза, отпечаток пальца). Мобильный интерфейс для работы охранников с журналом событий.

ЛИТЕРАТУРА

1. Официальная документация MongoDB. — URL: <https://www.mongodb.com/docs/>
2. Документация FastAPI. — URL: <https://fastapi.tiangolo.com/>
3. Документация PyMongo / Motor. — URL: <https://pymongo.readthedocs.io/>
4. Документация Docker Compose. — URL: <https://docs.docker.com/compose/>
5. Репозиторий проекта. — URL: <https://github.com/moevm/nsql1h26-bio>

ПРИЛОЖЕНИЯ

Приложение А: Документация по сборке и развёртыванию

Требования: Docker 24+, Docker Compose v2.

Порядок развёртывания:

```
# 1. Клонировать репозиторий
git clone https://github.com/moevm/nsql1h26-bio.git
cd nsql1h26-bio

# 2. Создать файл .env в корне проекта со следующим содержимым:
MONGO_URL=mongodb://db:27017
JWT_SECRET=your_secret_key

# 3. Собрать образы и запустить контейнеры
docker compose build --no-cache && docker compose up

# 4. Открыть приложение в браузере
# http://127.0.0.1:8080
```

При первом запуске backend автоматически обнаруживает пустую базу данных и загружает тестовые данные (не менее 7–10 объектов каждой сущности). Дополнительных шагов для наполнения базы не требуется.

Тестовые учётные записи:

Таблица 8 — Учётные данные тестовых пользователей

Логин	Пароль	Роль
admin	admin123	Администратор
guard	guard123	Охранник / Аудитор

Приложение Б: Инструкция для пользователя

Вход в систему:

11. Открыть браузер, перейти по адресу <http://127.0.0.1:8080>.
12. Ввести логин и пароль (по умолчанию: admin / admin).
13. Нажать «Войти». Система перенаправит на дашборд.

Создание пользователя:

14. Перейти в раздел «Пользователи» в боковом меню.
15. Нажать кнопку «Создать».
16. Заполнить поля формы, нажать «Сохранить».

Просмотр журнала событий:

- 17.Перейти в раздел «События».
- 18.Использовать поля фильтрации (зона, пользователь, дата, результат).
- 19.Для просмотра деталей нажать на строку события.

Экспорт данных:

- 20.Перейти в раздел «Обслуживание».
- 21.Нажать кнопку «Экспорт». Браузер скачает файл export.json.

Импорт данных:

- 22.Перейти в раздел «Обслуживание».
- 23.Выбрать JSON-файл через поле загрузки.
- 24.Нажать «Импорт». Существующие данные будут заменены данными из файла.

